BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 5

Roll no: 51 PRN No: 12411033 Name of Student: Manjiri Kulkarni

Subject Name: Deep Learning

Problem Statement:

Implement and compare RNN, LSTM and GRU models for sequence classification and analyze their performance using appropriate evaluation metrics.

Algorithm:

1. Start the program.
2. Import the required libraries and load the gold price dataset.
3. Preprocess the data by cleaning missing values and converting prices into numeric format.
4. Visualize the historical gold price trend.
5. Normalize the prices and create sequences using the previous 30 days.
6. Split the dataset into training and testing sets in an 80:20 ratio.
7. Build RNN, LSTM, and GRU models with 50 units and a Dense output layer.
8. Compile each model using the Adam optimizer and Mean Squared Error loss function.
9. Train each model for 10 epochs and predict gold prices.
10. Evaluate the models using MAE, RMSE, and R^2 score.
11. Plot actual versus predicted prices and compare the performance of all three models.
12. Stop the program.

Code:

```
import kagglehub
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, LSTM, GRU, Dense
path = kagglehub.dataset_download(
    "rizkykiky/gold-price-dataset"
)
print("Path to dataset files:", path)
print("Files in dataset folder:")
print(os.listdir(path))
df = pd.read_csv(
    os.path.join(path, "Daily.csv")
)
print(df.head())
print(df.columns)
print(df.shape)
df["Date"] = pd.to_datetime(df["Date"])
df = df[["Date", "INR"]]
df = df.dropna()
df = df.sort_values("Date")
# Remove commas and convert INR to numeric
df["INR"] = df["INR"].str.replace(",", "", regex=False)
df["INR"] = pd.to_numeric(
    df["INR"],
    errors="coerce"
)
df = df.dropna(subset=["INR"])
print(df.head())
print(df.shape)
plt.figure(figsize=(12, 5))
plt.plot(
    df["Date"],
    df["INR"]
)
plt.xlabel("Date")
plt.ylabel("Gold Price (INR)")
plt.title("Gold Price Trend")
plt.grid(True)
plt.show()
prices = df["INR"].values.reshape(-1, 1)
scaler = MinMaxScaler()
prices_scaled = scaler.fit_transform(prices)
print("Original price range:",
      prices.min(), "to", prices.max())
print("Scaled price range:",
      prices_scaled.min(), "to", prices_scaled.max())
```

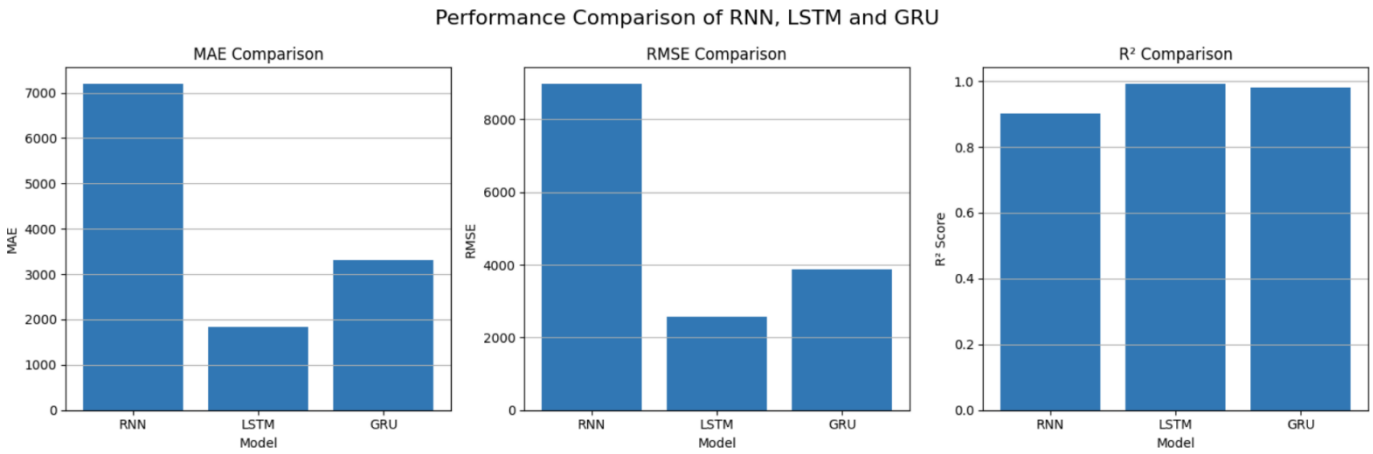
```
sequence_length = 30
X = []
y = []
for i in range(
    sequence_length,
    len(prices_scaled)
):
    X.append(
        prices_scaled[
            i-sequence_length:i
        ]
    )
    y.append(
        prices_scaled[i]
    )
X = np.array(X)
y = np.array(y)
print("X shape:", X.shape)
print("y shape:", y.shape)
train_size = int(len(X) * 0.8)
X_train = X[:train_size]
X_test = X[train_size:]
y_train = y[:train_size]
y_test = y[train_size:]
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
def build(kind):
    model = Sequential()
    if kind == "RNN":
        model.add(
            SimpleRNN(
                50,
                input_shape=(
                    X_train.shape[1],
                    X_train.shape[2]
                )
            )
        )
    elif kind == "LSTM":
        model.add(
            LSTM(
                50,
                input_shape=(
                    X_train.shape[1],
                    X_train.shape[2]
                )
            )
        )
    else:
        model.add(
            GRU(
```

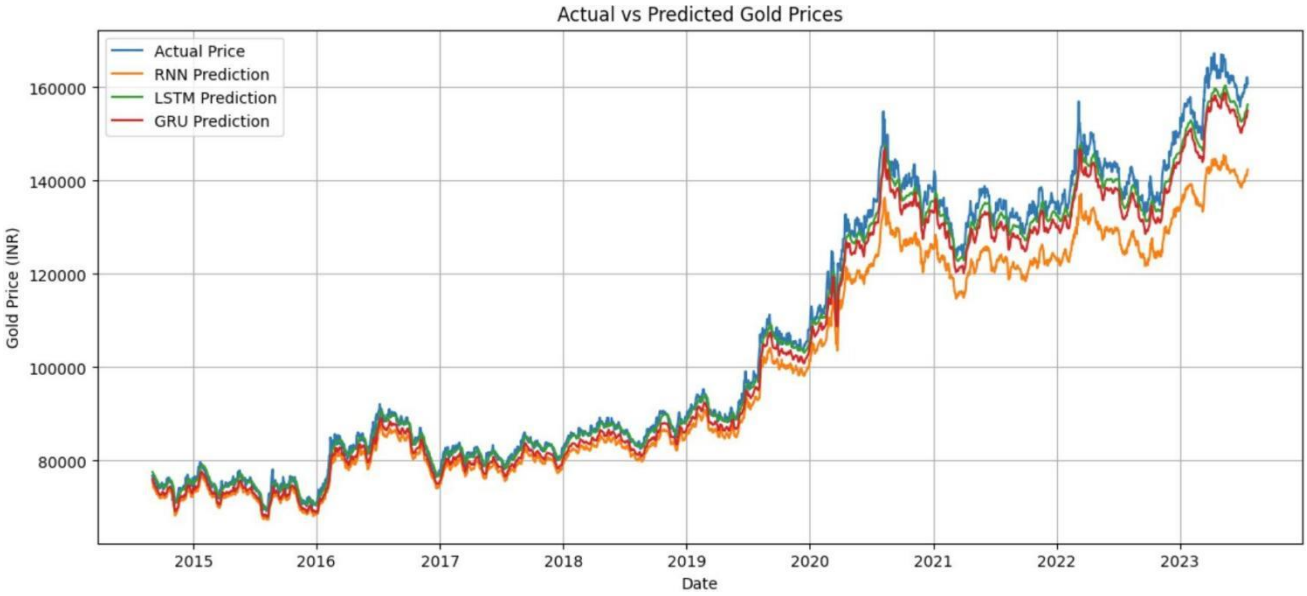
```
            50,
            input_shape=(
                X_train.shape[1],
                X_train.shape[2]
            )
        )
    )
    model.add(Dense(1))
    model.compile(
        optimizer="adam",
        loss="mean_squared_error"
    )
    return model
results = []
predictions_dict = {}
for kind in ["RNN", "LSTM", "GRU"]:
    print("\n")
    print(kind)
    model = build(kind)
    history = model.fit(
        X_train,
        y_train,
        epochs=10,
        batch_size=32,
        validation_split=0.2,
        verbose=1
    )
    predictions = model.predict(
        X_test,
        verbose=0
    )
    predicted_prices = scaler.inverse_transform(
        predictions
    )
    actual_prices = scaler.inverse_transform(
        y_test
    )
    predictions_dict[kind] = predicted_prices
    mae = mean_absolute_error(
        actual_prices,
        predicted_prices
    )
    rmse = np.sqrt(
        mean_squared_error(
            actual_prices,
            predicted_prices
        )
    )
    r2 = r2_score(
        actual_prices,
        predicted_prices
```

```
    )
    print("MAE:", mae)
    print("RMSE:", rmse)
    print("R2:", r2)
    results.append([
        kind,
        mae,
        rmse,
        r2
    ])
plt.figure(figsize=(14, 6))
plt.plot(
    test_dates,
    actual_prices.flatten(),
    label="Actual Price"
)
plt.plot(
    test_dates,
    predictions_dict["RNN"].flatten(),
    label="RNN Prediction"
)
plt.plot(
    test_dates,
    predictions_dict["LSTM"].flatten(),
    label="LSTM Prediction"
)
plt.plot(
    test_dates,
    predictions_dict["GRU"].flatten(),
    label="GRU Prediction"
)
plt.xlabel("Date")
plt.ylabel("Gold Price (INR)")
plt.title("Actual vs Predicted Gold Prices")
plt.legend()
plt.grid(True)
plt.show()
results_df = pd.DataFrame(
    results,
    columns=[
        "Model",
        "MAE",
        "RMSE",
        "R2"
    ]
)
print(results_df)
plt.figure(figsize=(15, 5))
# MAE
plt.subplot(1, 3, 1)
plt.bar(
```

```
    results_df["Model"],
    results_df["MAE"]
)
plt.title("MAE Comparison")
plt.xlabel("Model")
plt.ylabel("MAE")
plt.grid(axis="y")
# RMSE
plt.subplot(1, 3, 2)
plt.bar(
    results_df["Model"],
    results_df["RMSE"]
)
plt.title("RMSE Comparison")
plt.xlabel("Model")
plt.ylabel("RMSE")
plt.grid(axis="y")
# R2
plt.subplot(1, 3, 3)
plt.bar(
    results_df["Model"],
    results_df["R2"]
)
plt.title("R² Comparison")
plt.xlabel("Model")
plt.ylabel("R² Score")
plt.grid(axis="y")
plt.suptitle(
    "Performance Comparison of RNN, LSTM and GRU",
    fontsize=16
)
plt.tight_layout()
plt.show()
```

Output:




Actual vs Predicted Gold Prices
Actual Price
RNN Prediction
LSTM Prediction
GRU Prediction
Gold Price (INR)
160000
140000
120000
100000
80000
2015
2016
2017
2018
2019
2020
2021
2022
2023
Date